

University of Tehran
College of Engineering
School of Electrical and Computer
Engineering



Distributed Systems HW1

Part 2

Full Name
Sepanta Ghonoodi

Student ID
2111143200

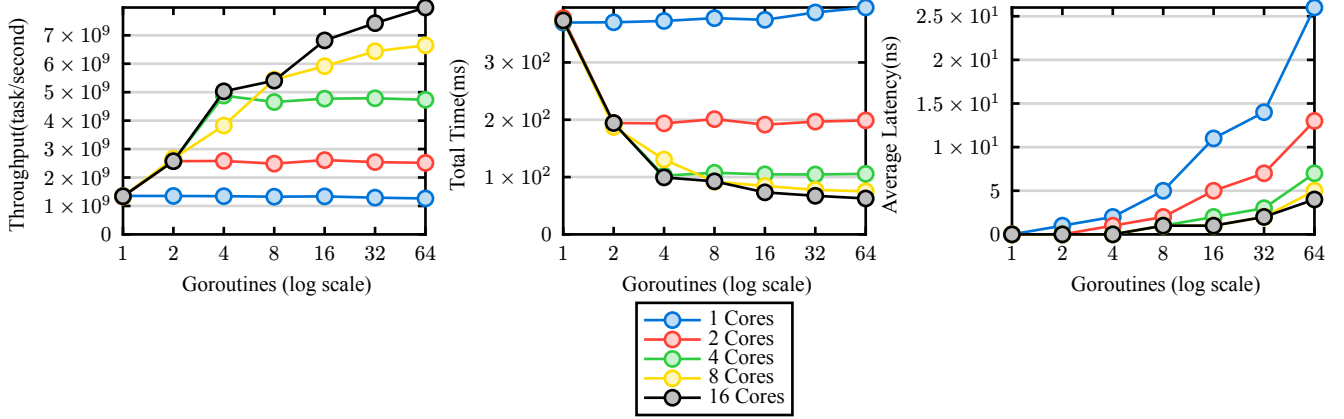
May 12, 2026

Table of Contents

CPU-Bound task:	2
Mutex Task	3
Sleep Task	4
Table of Results	4
Final Conclusions:	7
1. What happens to the execution time as the number of goroutines increases?	7
2. Does increasing the number of goroutines always increase throughput?	8
3. What is the difference in program behavior between GOMAXPROCS=1 and GOMAXPROCS=NumCPU?	8
4. In which type of workload are the effects of scheduling and context switching seen the most?	8
5. From what point does increasing concurrency become useless or even decrease performance?	8

CPU-Bound task:

In this section, we analyze a purely CPU-bound task that calculates a mathematical sum. The total computational workload remains constant across all tests, but it is divided into equal sub-parts and distributed among the active goroutines. Below, we present the total execution time, throughput, and average latency for each configuration.



This workload involves neither I/O waits (`sleep`) nor synchronization blocks (`mutex`). Consequently, when we increase the number of goroutines on a single physical core (`GOMAXPROCS=1`), we observe no improvement in throughput or total execution time. Instead, the average latency per task increases drastically. Because all goroutines are initialized simultaneously but forced to time-slice on a single core, they spend the majority of their lifecycle waiting in the scheduler's queue. While the absolute total execution time remains flat, the active lifespan of each individual worker is stretched, negatively impacting the average latency metric.

Furthermore, we observe that increasing the number of physical cores (`GOMAXPROCS > 1`) yields no performance boost if the number of goroutines remains lower than the available cores (e.g., 4 cores but 1 goroutine). In this state, the workload is not fully parallelized, leaving the additional physical CPU cores idle.

However, as the plots demonstrate, when multiple cores are available, increasing the number of goroutines creates a massive performance boost in both total time and throughput. This scaling continues optimally until the number of goroutines matches the number of physical cores. Beyond this peak—when goroutines exceed the available physical cores—we no longer see improvements. This proves that for purely computational workloads, adding more goroutines does not equate to better parallelization once hardware resources are fully saturated; it only reintroduces context-switching overhead.

Finally, we can see that even under optimal parallelization (adding both cores and goroutines), the average latency still trends upward. This occurs because parallelization improves overall throughput, but it does not reduce the execution time of an individual worker's batch. Since each worker measures its latency from the start of the application to its respective finish line, adding more concurrent workers inherently increases the average measured turnaround time of the tasks.

Mutex Task

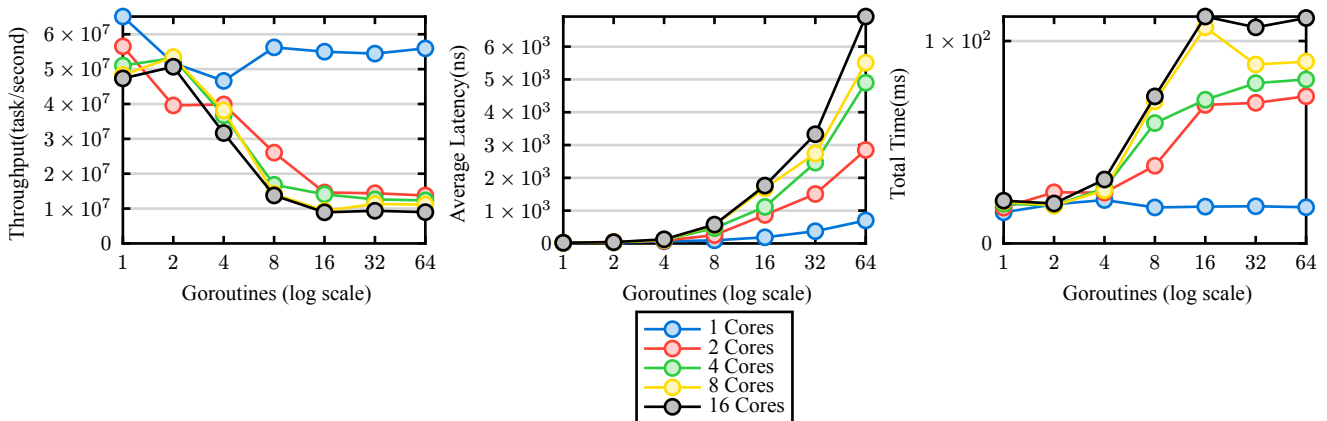
In this section, we examine a task constrained by a Mutex, where multiple goroutines attempt to increment a shared variable within a critical section.

When limited to a single physical core (GOMAXPROCS=1), increasing the number of goroutines does not trigger a catastrophic drop in performance. Because only one operating system thread executes at any given microsecond, the lock is acquired and released sequentially. Consequently, there is no true hardware-level contention for the memory address, keeping the throughput relatively stable.

However, when we increase the number of physical cores alongside the number of goroutines, we observe a severe degradation in performance. In a multi-core environment, parallel CPU cores actively compete to acquire the exact same lock to mutate the shared variable. This introduces massive hardware-level contention, as the cores constantly invalidate each other's caches.

Furthermore, this hardware contention compounds with software-level overhead. As multiple goroutines fail to acquire the lock, the Go scheduler is forced to spend a significant portion of its total execution time managing the traffic—spinning, parking (putting to sleep), and subsequently waking up blocked goroutines.

Therefore, for highly contended workloads, throwing more hardware resources (cores) and software concurrency (goroutines) at the problem does not improve throughput. Instead, the compounding overhead of cache contention and scheduler thrashing leads to a drastic loss of overall performance.



Sleep Task

In this final section, we introduce a time.Sleep element to the workload, simulating an I/O-bound task such as waiting for a network response or a database query.

When executing this workload with only a single goroutine, performance is exceptionally poor. Because the execution is entirely sequential, the single worker must initiate a task, enter a sleep state, and wait for the entire duration to pass before moving on to the next operation. The CPU remains completely idle while the thread is blocked, resulting in the longest total execution time and the lowest possible throughput.

However, by increasing the number of goroutines, we observe a massive improvement in performance, even when constrained to a single physical core (GOMAXPROCS=1). This demonstrates the core advantage of Go's concurrency model for I/O-bound tasks: latency hiding. When a goroutine enters a sleep state, it voluntarily yields the CPU. The Go scheduler immediately context-switches, placing a different, ready goroutine onto the core. By rapidly interleaving thousands of sleeping tasks, the scheduler ensures the physical CPU is never left waiting idly.

Counter-intuitively, the data reveals that assigning too many physical cores (e.g., GOMAXPROCS=16) to this highly concurrent workload actually degrades performance. Because the actual computational work executed between sleep cycles is microscopically small, the numerous physical cores quickly empty their local task queues. This forces the idle cores to aggressively scan other processors to steal work. This relentless "work-stealing" overhead, combined with the extreme hardware cost of synchronizing thousands of timer interrupts across multiple CPU caches, causes the scheduler to thrash.

Therefore, in contexts involving massive concurrency but minimal CPU execution (heavy I/O), maximizing physical hardware cores is actively detrimental. A moderate number of cores provides the optimal balance of processing power without triggering catastrophic administrative overhead.

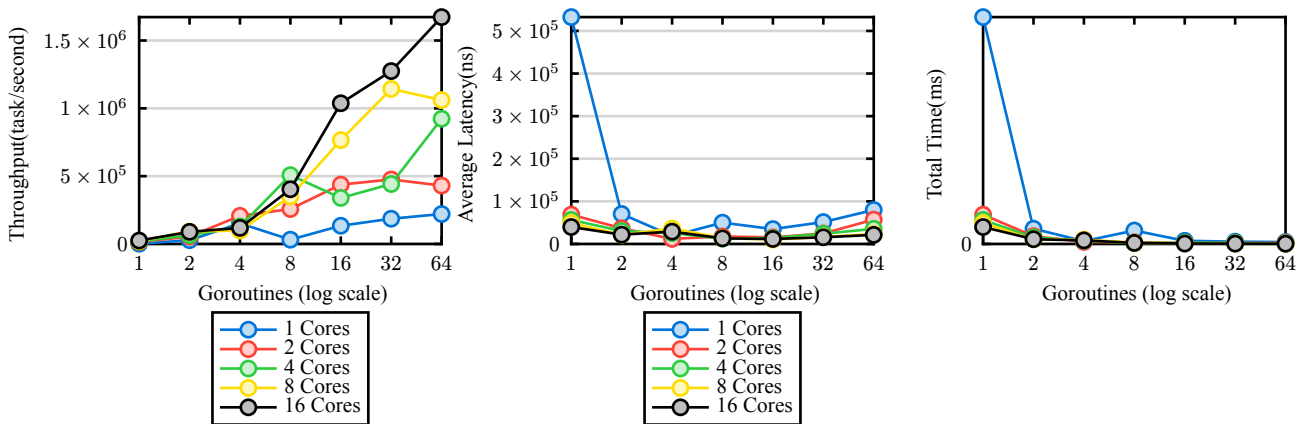


Table of Results

Here is the raw results of the code in the format of a table for each configuration.

Workload	GOMAXPROCS	Goroutines	TotalTime_ms	AvgLatency_ns	Throughput
CPU-bound	1	1	369.555	0	1352978813.00
CPU-bound	1	2	370.027	1	1351252045.28

CPU-bound	1	4	372.328	2	1342901492.14
CPU-bound	1	8	377.257	5	1325356556.50
CPU-bound	1	16	374.389	11	1335511106.78
CPU-bound	1	32	387.199	14	1291325352.27
CPU-bound	1	64	395.810	26	1263231031.29
CPU-bound	2	1	377.348	0	1325038433.73
CPU-bound	2	2	194.252	0	2573970957.56
CPU-bound	2	4	193.660	1	2581846402.81
CPU-bound	2	8	201.017	2	2487348215.23
CPU-bound	2	16	191.429	5	2611939417.47
CPU-bound	2	32	196.693	7	2542026950.26
CPU-bound	2	64	198.877	13	2514114009.78
CPU-bound	4	1	373.522	0	1338608933.93
CPU-bound	4	2	193.580	0	2582917141.87
CPU-bound	4	4	102.453	0	4880269898.45
CPU-bound	4	8	107.438	1	4653825384.90
CPU-bound	4	16	104.777	2	4772046314.73
CPU-bound	4	32	104.471	3	4786012407.79
CPU-bound	4	64	105.617	7	4734075924.90
CPU-bound	8	1	372.994	0	1340503148.47
CPU-bound	8	2	187.481	0	2666942449.85
CPU-bound	8	4	130.659	0	3826744304.68
CPU-bound	8	8	91.876	1	5442104190.89
CPU-bound	8	16	84.545	1	5913992382.97
CPU-bound	8	32	77.656	2	6438625489.44
CPU-bound	8	64	75.208	5	6648221309.61
CPU-bound	16	1	373.249	0	1339588845.30
CPU-bound	16	2	194.270	0	2573731606.77
CPU-bound	16	4	99.418	0	5029264080.66
CPU-bound	16	8	92.490	1	5406018535.57
CPU-bound	16	16	73.313	1	6820053023.73
CPU-bound	16	32	67.321	2	7427108390.08
CPU-bound	16	64	62.678	4	7977275359.05
Mutex-bound	1	1	15.366	15	65080134.47
Mutex-bound	1	2	19.321	31	51756485.73
Mutex-bound	1	4	21.447	64	46627535.28
Mutex-bound	1	8	17.781	94	56240037.43
Mutex-bound	1	16	18.180	187	55004148.14
Mutex-bound	1	32	18.364	372	54454281.25

Mutex-bound	1	64	17.872	701	55953215.05
Mutex-bound	2	1	17.689	17	56533858.27
Mutex-bound	2	2	25.258	45	39591319.40
Mutex-bound	2	4	25.084	83	39865953.12
Mutex-bound	2	8	38.394	254	26045608.10
Mutex-bound	2	16	68.419	869	14615909.61
Mutex-bound	2	32	69.460	1507	14396714.81
Mutex-bound	2	64	72.693	2849	13756484.07
Mutex-bound	4	1	19.597	19	51028658.66
Mutex-bound	4	2	18.801	37	53189407.05
Mutex-bound	4	4	27.220	107	36737242.09
Mutex-bound	4	8	59.457	466	16818813.30
Mutex-bound	4	16	70.991	1114	14086362.47
Mutex-bound	4	32	79.177	2462	12629941.73
Mutex-bound	4	64	81.016	4901	12343281.52
Mutex-bound	8	1	20.701	20	48307358.47
Mutex-bound	8	2	18.705	36	53461755.60
Mutex-bound	8	4	26.220	102	38139034.78
Mutex-bound	8	8	70.189	558	14247221.55
Mutex-bound	8	16	106.805	1687	9362824.58
Mutex-bound	8	32	88.391	2742	11313426.48
Mutex-bound	8	64	89.841	5510	11130794.44
Mutex-bound	16	1	21.126	21	47336106.19
Mutex-bound	16	2	19.729	39	50685673.25
Mutex-bound	16	4	31.575	125	31670665.62
Mutex-bound	16	8	72.636	573	13767217.28
Mutex-bound	16	16	112.086	1763	8921697.66
Mutex-bound	16	32	106.763	3329	9366528.30
Mutex-bound	16	64	111.375	6917	8978712.73
Sleep-bound	1	1	2662.349	532462	1878.04
Sleep-bound	1	2	179.059	70338	27923.76
Sleep-bound	1	4	32.884	18971	152048.61
Sleep-bound	1	8	157.942	50050	31657.20
Sleep-bound	1	16	37.237	34939	134276.38
Sleep-bound	1	32	26.868	51384	186095.65
Sleep-bound	1	64	22.656	79856	220692.81
Sleep-bound	2	1	344.110	68817	14530.22
Sleep-bound	2	2	91.701	36669	54525.10
Sleep-bound	2	4	24.096	11996	207505.15

Sleep-bound	2	8	19.416	17539	257519.61
Sleep-bound	2	16	11.426	15510	437588.81
Sleep-bound	2	32	10.511	24658	475712.82
Sleep-bound	2	64	11.618	57126	430374.27
Sleep-bound	4	1	282.268	56448	17713.68
Sleep-bound	4	2	77.312	30450	64672.99
Sleep-bound	4	4	38.856	25677	128678.78
Sleep-bound	4	8	9.850	12611	507600.25
Sleep-bound	4	16	14.733	13727	339382.16
Sleep-bound	4	32	11.310	24003	442082.47
Sleep-bound	4	64	5.414	35534	923463.53
Sleep-bound	8	1	244.311	48857	20465.75
Sleep-bound	8	2	54.502	21560	91739.20
Sleep-bound	8	4	50.694	36366	98631.40
Sleep-bound	8	8	14.297	13764	349731.13
Sleep-bound	8	16	6.529	10351	765789.78
Sleep-bound	8	32	4.374	16478	1143108.50
Sleep-bound	8	64	4.713	23382	1060893.14
Sleep-bound	16	1	199.000	39794	25125.57
Sleep-bound	16	2	55.925	21831	89405.97
Sleep-bound	16	4	42.181	28908	118536.75
Sleep-bound	16	8	12.398	13141	403290.46
Sleep-bound	16	16	4.820	11698	1037333.42
Sleep-bound	16	32	3.923	15661	1274606.92
Sleep-bound	16	64	2.989	21343	1672768.37

\

Final Conclusions:

1. What happens to the execution time as the number of goroutines increases?

CPU-Bound:

On a single core, increasing goroutines does not decrease execution time; it actually inflates the average latency per task because the scheduler must time-slice the workload, forcing tasks to wait in a queue. On multiple cores, execution time decreases linearly until the number of goroutines equals the number of physical cores.

Mutex-Bound:

Increasing goroutines drastically increases the execution time. The overhead of context switching, lock thrashing, and parking/waking blocked goroutines dominates the CPU time, slowing down the actual computation.

Sleep-Bound:

Execution time decreases significantly as goroutines increase, because the Go scheduler can effectively hide the latency of sleeping tasks by continuously swapping in ready goroutines.

2. Does increasing the number of goroutines always increase throughput?

No, increasing goroutines only increases throughput when there are idle hardware resources or when tasks are waiting on I/O.

In purely computational tasks limited to one core, adding goroutines slightly decreases throughput due to context-switching overhead.

In highly contended scenarios (Mutex locks), adding goroutines destroys throughput, as the system spends more time managing access to the lock than executing the critical section.

3. What is the difference in program behavior between GOMAXPROCS=1 and GOMAXPROCS=NumCPU?

GOMAXPROCS=1 (Concurrency without Parallelism): Forces the Go runtime to execute all goroutines sequentially on a single operating system thread. This eliminates hardware-level race conditions and cache invalidation issues, making heavily contended tasks (like a Mutex lock) run surprisingly fast, but limits computational scaling.

GOMAXPROCS=NumCPU (True Parallelism): Spreads goroutines across all physical CPU cores. This provides massive speedups for parallelizable, independent computations (CPU-bound) and I/O tasks. However, it exposes shared states to severe hardware-level contention, causing cores to constantly invalidate each other's caches when fighting for the same memory address.

4. In which type of workload are the effects of scheduling and context switching seen the most?

Positive Effect (Sleep-Bound): The scheduler's ability to instantly context-switch when a goroutine enters a sleep state is what allows a single core to process thousands of concurrent tasks efficiently.

Negative Effect (Mutex-Bound): The scheduler is forced to rapidly switch, spin, park, and wake goroutines that are fighting over a locked resource, resulting in severe "scheduler thrashing" that degrades performance.

5. From what point does increasing concurrency become useless or even decrease performance?

Increasing concurrency(goroutines) degrades performance at two distinct thresholds:

- **Hardware Saturation:** For purely independent, CPU-bound tasks, concurrency becomes useless the moment the number of goroutines exceeds the number of physical cores (GOMAXPROCS). Any additional goroutines simply add time-slicing overhead without adding processing power.
- **State Contention:** For tasks sharing memory, concurrency decreases performance the moment multiple active threads attempt to mutate the exact same state simultaneously. As the ratio of "waiting goroutines" to "active locks" increases, the system falls into lock thrashing, drastically reducing overall efficiency.